

Assignment-18.4

Name:N.Soumya
H NO:2303A51869
Batch-13

Task 1 – Movie Database API Integration

Prompt: Create a Python script using the requests library to fetch movie details from OMDb API. Requirements: Take movie title as user input, Fetch data from API using API key, Display: Title, Year, IMDb Rating, Plot, Handle errors: invalid movie, API key issues, network errors, empty response, Use clean formatting and functions for better readability.

Code:

```
ass-18.4.py > get_movie_details
1 #Create a Python script using the requests library to fetch movie
2 #details from OMDb API. Requirements: Take movie title as user input, Fetch
3 #data from API using API key, Display: Title, Year, IMDb Rating, Plot, Handle
4 #errors: invalid movie, API key issues, network errors, empty response, Use clean
5 #formatting and functions for better readability.
6 import requests
7 API_KEY = "d60f7ff3"
8 BASE_URL = "http://www.omdbapi.com/"
9 def get_movie_details(movie_title):
10     params = {
11         "apikey": API_KEY,
12         "t": movie_title
13     }
14     try:
15         response = requests.get(BASE_URL, params=params)
16         response.raise_for_status() # Raise an error for HTTP errors
17         data = response.json()
18         if data.get("Response") == "False":
19             print("Movie not found.")
20             return None
21         return data
22     except requests.RequestException as e:
23         print(f"Error fetching movie details: {e}")
24         return None
25 def display_movie_details(movie_data):
26     if not movie_data:
27         return
28     print(f"Title: {movie_data.get('Title', 'N/A')}")
29     print(f"Year: {movie_data.get('Year', 'N/A')}")
30     print(f"IMDb Rating: {movie_data.get('imdbRating', 'N/A')}")
31     print(f"Plot: {movie_data.get('Plot', 'N/A')}")
32 def main():
33     movie_title = input("Enter the movie title: ")
34     movie_data = get_movie_details(movie_title)
35     display_movie_details(movie_data)
36 if __name__ == "__main__":
37     main()
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> & C:\Users\vempa\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING\ass-18.4.py"
Enter the movie title: Guardians of the Galaxy Vol. 2
Title: Guardians of the Galaxy Vol. 2
Year: 2017
IMDb Rating: 7.6
Plot: The Guardians struggle to keep together as a team while dealing with their personal family issues, notably Star-Lord's encounter with his father, the ambitious celestial being Ego.
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING>
```

Explanation : This script allows users to fetch movie details from the OMDb API by entering a movie title and an API key. It uses the requests library to make an HTTP GET request to the API, retrieves the movie details in JSON format, and displays the title, year, IMDb rating, and plot. The script also includes error handling for invalid movie titles, API key issues, network errors, and empty responses.

Task 2 – Public Transport Schedule API

Prompt :

Create a Python script using the requests library to fetch public transport arrival times from an API. Take stop ID as user input, validate it, and display the next 3 arrivals. Handle errors like invalid stop ID, server issues, timeout, and empty response using try-except. Include at least one retry attempt and keep the output clean and readable.

Code:

```
#Create a Python script using the requests library to fetch public transport arrival times from an API.
# Take stop ID as user input, validate it, and display the next 3 arrivals. Handle errors like invalid
# stop ID, server issues, timeout, and empty response using try-except. Include at least one retry attempt
# and keep the output clean and readable.
#give code
import requests
import time
from datetime import datetime

API_URL = "https://api.tfl.gov.uk/StopPoint/{stop_id}/Arrivals"

def fetch_arrival_times(stop_id):
    url = API_URL.format(stop_id=stop_id)

    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        data = response.json()

        # Tfl returns a list, not a dict
        if not isinstance(data, list) or len(data) == 0:
            print("▲ No arrivals found for this stop.")
            return None

        # Sort by arrival time
        data.sort(key=lambda x: x.get("timeToStation", 999999))

        return data[:3] # next 3 arrivals

    except requests.exceptions.HTTPError as e:
        print("✗ Invalid Stop ID or API issue:", e)

    except requests.exceptions.ConnectionError:
        print("✗ Network error. Check your internet connection.")

    except requests.exceptions.Timeout:
```

```

ass-18.4.py > ...
51 def fetch_arrival_times(stop_id):
52     """Fetch arrival times for a given stop ID"""
53     try:
54         response = requests.get(f"https://api.tfl.gov.uk/stop/{stop_id}/arrivals")
55         if response.status_code == 200:
56             return response.json()
57         else:
58             print(f"Error: {response.status_code}")
59             return None
60     except requests.exceptions.Timeout:
61         print("⌛ Request timed out.")
62     except requests.exceptions.RequestException as e:
63         print("⌛ Unexpected error:", e)
64     return None
65
66 def format_time(seconds):
67     """Format time in seconds to minutes and seconds"""
68     minutes = seconds // 60
69     seconds = seconds % 60
70     return f"{minutes} min {seconds} sec"
71
72 def display_arrivals(arrivals):
73     """Display arrival times for a given stop ID"""
74     if not arrivals:
75         print("🚫 No data to display.")
76         return
77
78     print("\n🚌 Next Bus Arrivals")
79     print("-" * 40)
80
81     for bus in arrivals:
82         line = bus.get("lineName", "N/A")
83         destination = bus.get("destinationName", "Unknown")
84         time_to_arrival = format_time(bus.get("timeToStation", 0))
85
86         print(f"🚌 Bus {line} → {destination}")
87         print(f"    Arrival in: {time_to_arrival}")
88         print("-" * 40)
89
90 def main():
91     """Main function to run the program"""
92     print("👋 Welcome to Smart Bus Arrival Checker")
93
94     stop_id = input("Enter Tfl Stop ID (e.g., 490008660N): ").strip()
95     if not stop_id:
96         print("⌛ Stop ID cannot be empty.")
97         return
98
99     arrivals = fetch_arrival_times(stop_id)
100     if arrivals is None:
101         print("⌛ Retrying in 2 seconds...")
102         time.sleep(2)
103         arrivals = fetch_arrival_times(stop_id)
104
105     display_arrivals(arrivals)
106
107 if __name__ == "__main__":
108     main()

```

Output:

```

ass-18.4.py > main
107 def main():
108     print("👋 Welcome to Smart Bus Arrival Checker")
109     stop_id = input("Enter Tfl Stop ID (e.g., 490008660N): ").strip()
110     if not stop_id:
111         print("⌛ Stop ID cannot be empty.")
112         return
113     arrivals = fetch_arrival_times(stop_id)
114     if arrivals is None:
115         print("⌛ Retrying in 2 seconds...")
116         time.sleep(2)
117         arrivals = fetch_arrival_times(stop_id)
118
119     display_arrivals(arrivals)
120 if __name__ == "__main__":
121     main()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> C:\Users\vempa\AppData\Local\Programs\Python\Python311\python.exe C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING\ass-18.4.py

👋 Welcome to Smart Bus Arrival Checker

Enter Tfl Stop ID (e.g., 490008660N): 490008660N

🚌 Next Bus Arrivals

🚌 Bus 88 → Parliament Hill Fields
Arrival in: 2 min

🚌 Bus 88 → Parliament Hill Fields
Arrival in: 10 min

🚌 Bus 88 → Parliament Hill Fields
Arrival in: 23 min

Explanation:

The program takes a stop ID as input and fetches arrival data from the TfL API using the requests library. It processes the response, sorts arrivals by nearest time, and displays the next 3 arrivals with details. It also handles errors (timeout, connection, invalid data) and includes a retry mechanism for reliability.

TASK – 03

Prompt : Create a Python script using the requests library to fetch live stock data from a financial API (such as Alpha Vantage). The program should accept a stock symbol as user input and display the current price, day high, and day low. It must handle errors like invalid stock symbols, API rate limits, and missing fields in the JSON response. Ensure proper validation and safe parsing of the API response using try-except blocks, and display clean, user-friendly output without any crashes.

Code:

```
import requests

API_KEY = "YOUR_API_KEY" # Replace with your Alpha Vantage API key
BASE_URL = "https://www.alphavantage.co/query"

def fetch_stock_data(symbol):
    if not symbol.strip():
        print("✗ Stock symbol cannot be empty.")
        return None

    params = {
        "function": "GLOBAL_QUOTE",
        "symbol": symbol.upper().strip(),
        "apikey": API_KEY
    }

    try:
        response = requests.get(BASE_URL, params=params, timeout=5)
        response.raise_for_status()
        data = response.json()

        # Handle API rate limit
        if "Note" in data:
            print("⚠ API limit reached. Please try again later.")
            return None

        # Handle invalid API key
        if "Error Message" in data:
```

```

    print("❌ Invalid API key or request.")
    return None

    quote = data.get("Global Quote")

    if not quote:
        print("❌ Invalid stock symbol or no data found.")
        return None

    return quote

except requests.exceptions.ConnectionError:
    print("❌ Network error. Check your internet connection.")

except requests.exceptions.Timeout:
    print("❌ Request timed out. Try again.")

except requests.exceptions.RequestException as e:
    print("❌ Unexpected error:", e)

return None

def display_stock_data(quote):
    try:
        symbol = quote.get("01. symbol", "N/A")
        price = quote.get("05. price", "N/A")
        high = quote.get("03. high", "N/A")
        low = quote.get("04. low", "N/A")

```

```

    print("\n📊 Stock Details")
    print("-" * 40)
    print(f"Symbol       : {symbol}")
    print(f"Current Price: ${price}")
    print(f"Day High      : ${high}")
    print(f"Day Low       : ${low}")
    print("-" * 40)

except Exception:
    print("❌ Error displaying stock data (missing fields).")

def main():
    print("🟢 Live Stock Price Checker")

    symbol = input("Enter stock symbol (e.g., AAPL, TSLA): ")

    stock_data = fetch_stock_data(symbol)

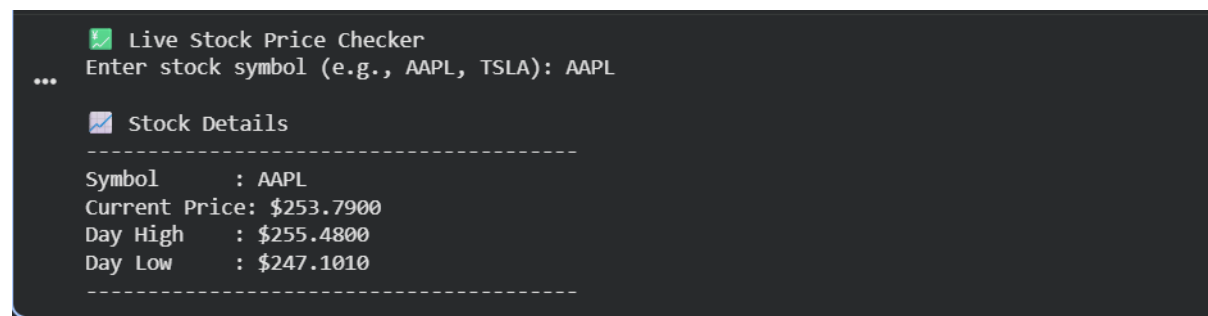
    if stock_data:
        display_stock_data(stock_data)
    else:
        print("⚠️ Unable to retrieve stock data.")

if __name__ == "__main__":
    main()

```

Output:

```



```

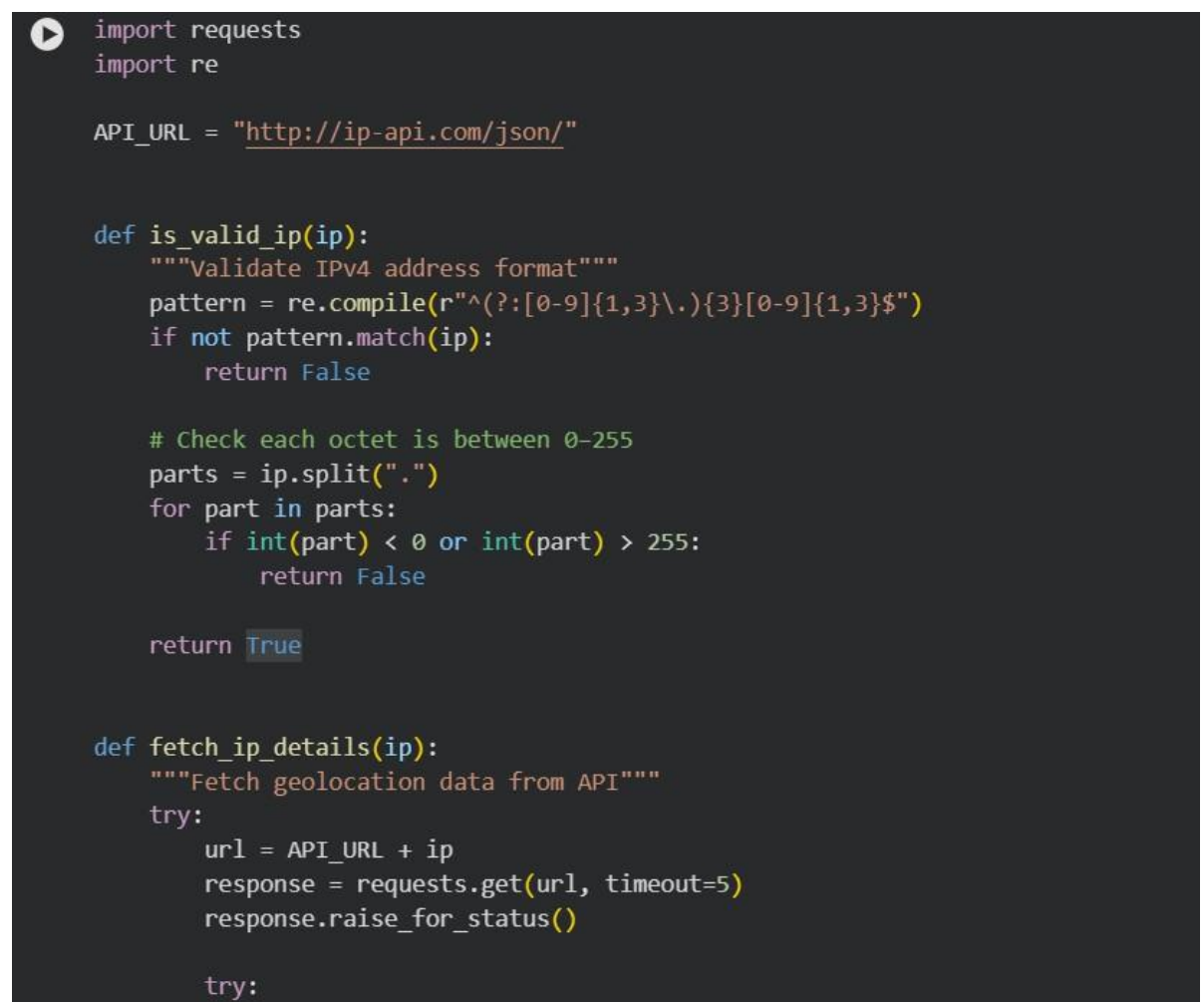
Explanation: The script takes a stock symbol as input and fetches its current price, day high, and day low from a financial API. It safely handles errors like invalid symbols, rate limits, and missing data, ensuring clean output without crashes.

TASK – 04:

Prompt : Generate a Python script that takes an IP address as input, validates its format, and uses a Geolocation API (like ip-api.com) to fetch country, city, and ISP. Include error handling for invalid IPs, API failures, and JSON parsing issues, and display results clearly.

Code:

```



```

```

        data = response.json()
    except ValueError:
        print("❌ Error parsing server response (invalid JSON).")
        return None

    # API-specific error handling
    if data.get("status") == "fail":
        print(f"❌ API Error: {data.get('message', 'Invalid IP address')}")
        return None

    return data

except requests.exceptions.ConnectionError:
    print("❌ Network error. Please check your internet connection.")

except requests.exceptions.Timeout:
    print("❌ Request timed out. Try again later.")

except requests.exceptions.HTTPError as e:
    print("❌ HTTP error:", e)

except requests.exceptions.RequestException as e:
    print("❌ Unexpected error:", e)

return None

def display_ip_details(data):
    """Display results cleanly"""
    if not data:

```

```

if not data:
    print("⚠ No data to display.")
    return

print("\n🌐 IP Geolocation Details")
print("-" * 40)
print(f"IP Address : {data.get('query', 'N/A')}")
print(f"Country   : {data.get('country', 'N/A')}")
print(f"City       : {data.get('city', 'N/A')}")
print(f"ISP        : {data.get('isp', 'N/A')}")
print("-" * 40)

def main():
    print("🌐 IP Geolocation Finder")

    ip = input("Enter an IP address: ").strip()

    if not is_valid_ip(ip):
        print("❌ Invalid IP address format.")
        return

    data = fetch_ip_details(ip)

    if data:
        display_ip_details(data)
    else:
        print("⚠ Could not retrieve IP details.")

```

Output:

```

if __name__ == "__main__":
    main()

```

... 🌐 IP Geolocation Finder
Enter an IP address: 8.8.8.8

🌐 IP Geolocation Details

```

-----
IP Address : 8.8.8.8
Country    : United States
City       : Ashburn
ISP        : Google LLC
-----

```

Explanation : This script takes an IP address, checks if it is valid, and then calls a Geolocation API to get details like country, city, and ISP. It also handles errors such as invalid IP format, API failure, and incorrect JSON response to ensure the program runs safely.

TASK – 05

Prompt : Generate a Python script that simulates payment processing using a sandbox Payment API. It should send a POST request with amount, currency, and payment method, validate inputs, handle errors (invalid details, HTTP errors, network issues), display transaction status, and log failed transactions to a file.

Code:

```
124 import requests
125 import json
126 from datetime import datetime
127
128 # Example sandbox endpoint (mock API)
129 API_URL = "https://jsonplaceholder.typicode.com/posts"
130
131
132 def validate_payment(amount, currency, method):
133     if amount <= 0:
134         print("❌ Amount must be greater than 0.")
135         return False
136
137     if not currency.isalpha() or len(currency) != 3:
138         print("❌ Currency must be a 3-letter code (e.g., USD, INR).")
139         return False
140
141     valid_methods = ["card", "upi", "netbanking"]
142     if method.lower() not in valid_methods:
143         print(f"❌ Invalid payment method. Choose from {valid_methods}")
144         return False
145
146     return True
147
148
149 def log_failed_transaction(data, error_message):
150     with open("failed_transactions.log", "a") as file:
151         log_entry = {
152             "timestamp": str(datetime.now()),
153             "data": data,
154             "error": error_message
155         }
156         file.write(json.dumps(log_entry) + "\n")
157
158
159 def process_payment(amount, currency, method):
```

```

ass-18.4.py > ...
159 def process_payment(amount, currency, method):
160     payload = {
161         "amount": amount,
162         "currency": currency.upper(),
163         "payment_method": method.lower()
164     }
165
166     try:
167         response = requests.post(API_URL, json=payload, timeout=5)
168         response.raise_for_status()
169
170         try:
171             data = response.json()
172         except ValueError:
173             print("❌ Invalid JSON response from server.")
174             log_failed_transaction(payload, "JSON parsing error")
175             return None
176
177         return data
178
179     except requests.exceptions.HTTPError as e:
180         print("❌ HTTP error:", e)
181         log_failed_transaction(payload, str(e))
182
183     except requests.exceptions.ConnectionError:
184         print("❌ Network error. Check your connection.")
185         log_failed_transaction(payload, "Connection error")
186
187     except requests.exceptions.Timeout:
188         print("❌ Request timed out.")
189         log_failed_transaction(payload, "Timeout")
190
191     except requests.exceptions.RequestException as e:
192         print("❌ Unexpected error:", e)
193         log_failed_transaction(payload, str(e))
194
195     return None

```

```

ass-18.4.py > ...
198 def display_result(response_data):
199     if not response_data:
200         print("⚠️ Payment failed.")
201         return
202
203     print("\n📄 Payment Status")
204     print("-" * 40)
205     print(f"Status      : SUCCESS ✅")
206     print(f"Transaction ID: {response_data.get('id', 'N/A')}")
207     print("-" * 40)
208
209
210 def main():
211     print("💰 Payment Processing Simulator")
212
213     try:
214         amount = float(input("Enter amount: "))
215     except ValueError:
216         print("❌ Invalid amount.")
217         return
218
219     currency = input("Enter currency (e.g., USD, INR): ").strip()
220     method = input("Enter payment method (card/upi/netbanking): ").strip()
221
222     if not validate_payment(amount, currency, method):
223         return
224
225     response = process_payment(amount, currency, method)
226     display_result(response)
227
228
229 if __name__ == "__main__":
230     main()

```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> & C:/Users/vempa/AppData/Local/Programs/Python/Python31
ctures/Desktop/HCL LAB/AI ASSISTANT CODING/ass-18.4.py"
🔥 Payment Processing Simulator
Enter amount: 500
Enter currency (e.g., USD, INR): INR
Enter payment method (card/upi/netbanking): upi

Enter currency (e.g., USD, INR): INR
Enter payment method (card/upi/netbanking): upi

📄 Payment Status
-----
Status      : SUCCESS ✅
Transaction ID: 101
-----
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> █
```

Explanation :

This script sends payment details (amount, currency, method) to a sandbox API, checks if inputs are valid, and processes the transaction. It handles errors like invalid data, server issues, or network failures, shows the payment status, and logs failed transactions to a file.